ATLAS
TRADING PLATFORM

ATLAS

Enterprise AI-Powered Algorithmic Trading Platform

ARCHITECTURE & DESIGN DOCUMENT

Version 0.1.0 // 2026-07-08

MT5-NATIVE · MULTI-TENANT SaaS · AI-NATIVE · CLOUD-READY

"The backend is the brain. AI modules are the analysts. The strategy engine is the decision-maker. The risk engine is the guardian. The MT5 Bridge is the messenger. MT5 is simply an execution adapter."

Table of Contents

1. Executive Summary

ATLAS is a production-grade, multi-tenant, AI-native algorithmic trading platform designed for long-term commercial deployment as a SaaS product. The platform supports thousands of users, hundreds of MT5 terminals across multiple brokers, and thousands of simultaneously active strategies — all governed by a centralized risk engine and orchestrated by a heterogeneous mix of specialized AI modules. Unlike typical trading bots that tightly couple strategy logic to a specific broker API, ATLAS treats every execution venue as a pluggable adapter, allowing the core platform to remain unchanged when new brokers, FIX gateways, or cryptocurrency exchanges are added.

The existing infrastructure — Ubuntu 22.04 with Wine, MetaTrader 5, Xvfb, Fluxbox, x11vnc, noVNC, systemd, Nginx, Docker, PostgreSQL, and Python 3.13 — is treated as a fixed runtime environment. Wine is regarded as an operating system dependency, not an application to be managed by the backend. The backend never shells out to `wine` or `mt5.exe`; all communication with MetaTrader 5 happens through a dedicated Bridge Service that exchanges versioned JSON messages over WebSocket with a small MQL5 Expert Advisor running inside each terminal. This indirection is the single most important architectural decision in the system — it enables horizontal scaling, multi-broker support, and complete decoupling between the platform brain and the execution layer.

The platform is built on clean architecture principles: domain-driven design for the core trading, risk, strategy, market-data, AI, and identity bounded contexts; the repository pattern for persistence; command-query responsibility segregation (CQRS) for the application layer; dependency injection for testability; and an event-driven backbone powered by Redis pubsub for in-process fanout and Celery for durable background work. Every module is independently deployable, every external dependency is hidden behind an interface, and every cross-cutting concern — logging, metrics, tracing, authentication, rate limiting — is centralized in the `platform.core` package. The result is a system that can be reasoned about, tested in isolation, and extended without triggering cascading changes.

Performance targets

- **10,000+ concurrent users** behind the REST API, with sub-50ms p99 latency on hot paths
- **1,000+ active strategies** evaluated on every closed bar across all subscribed symbols
- **Hundreds of MT5 terminals** connected to a single Bridge Service; horizontal scaling via sharding
- **Millions of stored ticks** with TimescaleDB-ready schema; partitioning by month in production
- **Zero-downtime deployments** via blue/green Docker Compose + bridge drain-and-reconnect

2. Design Philosophy

Every architectural decision in ATLAS flows from a small set of invariants. These are not aspirations — they are rules. Code that violates them gets rejected in review.

2.1 The brain / adapter separation

The backend is the brain. It owns the domain model, the strategy engine, the risk engine, the AI orchestrator, and the event bus. It never executes trades directly; it issues commands to execution adapters. Today the only production adapter is the MT5 Bridge; tomorrow it may be a FIX gateway, a crypto exchange REST+WS adapter, or a paper broker for backtesting. Adding a new adapter means implementing a single Python interface (`ExecutionAdapter`) and registering it with the plugin loader — no core code changes.

2.2 Wine is an OS dependency

Wine exists on the host to allow MetaTrader 5 (a Windows binary) to run on Linux. The backend never manages Wine. It never restarts Wine, never inspects Wine processes, never reads Wine logs. The MT5 terminal runs continuously inside its Wine prefix and registers itself with the backend at startup. If Wine crashes, systemd restarts it; the backend simply notices that the terminal went offline and recovers gracefully when it comes back. This separation eliminates an entire class of fragility that plagues amateur trading systems.

2.3 Clean architecture + DDD

The codebase is organized in concentric layers. The domain layer (`platform/domain`) contains pure business logic — aggregates, entities, value objects, and domain events — with zero imports from infrastructure packages. The application layer (`platform/application`) contains use-case handlers that orchestrate domain objects and repositories. The infrastructure layer (`platform/infrastructure`) provides concrete implementations of repository protocols, the MT5 Bridge client, Redis adapters, and Celery workers. The API layer (`platform/api`) is a thin translation surface between HTTP/WebSocket and the application layer. Dependencies always point inward; the domain never depends on infrastructure.

2.4 Event-driven where it matters

ATLAS uses an event-driven architecture for the hot path: every tick streamed by a terminal is published to the `atlas.ticks` topic on Redis pubsub, and every execution report is published to `atlas.execution_reports`. Subscribers — the market-data engine, strategy engine, AI orchestrator, risk engine, and WebSocket fan-out — consume these events independently. This decoupling lets the system scale horizontally: additional subscribers can be added without modifying the publisher, and a slow consumer cannot block the hot path because each subscriber runs in its own asyncio task or Celery worker.

2.5 Multi-tenant from day one

Every database table that owns user data carries an `org_id` column and is indexed on it. Every query path goes through a dependency-injected `CurrentUser` object that carries the caller's organization ID. This is not a bolted-on multi-tenancy layer; it is the structural default. New features inherit tenancy automatically because they reuse the same base classes and query patterns.

2.6 Pluggable everything

Strategies, AI modules, risk rules, execution adapters, notification channels, and future broker integrations are all plugins. Each plugin type has a base class / protocol and a registry; adding a new plugin is a single decorator or registration call. The platform ships with a curated set of built-ins (EMA cross, RSI reversion, SMC order blocks for strategies; Trend, Risk, Pattern AI for AI; KillSwitch, MaxDailyLoss, MaxDrawdown for risk; email, Telegram, Discord, webhook for notifications) and an explicit extension point for user-supplied plugins loaded from the `platform.plugins` namespace.

3. System Architecture

ATLAS is deployed as a set of cooperating processes on a single Ubuntu 22.04 host in the initial release, with a clear scaling path to multi-host and Kubernetes. The topology is intentionally simple: an Nginx edge that terminates TLS and routes to the FastAPI backend (REST + WebSocket), a Bridge Service that owns all MT5 terminal connections, PostgreSQL for durable state, Redis for cache and pubsub, and Celery workers for background work. Each process is independently restartable and observable.

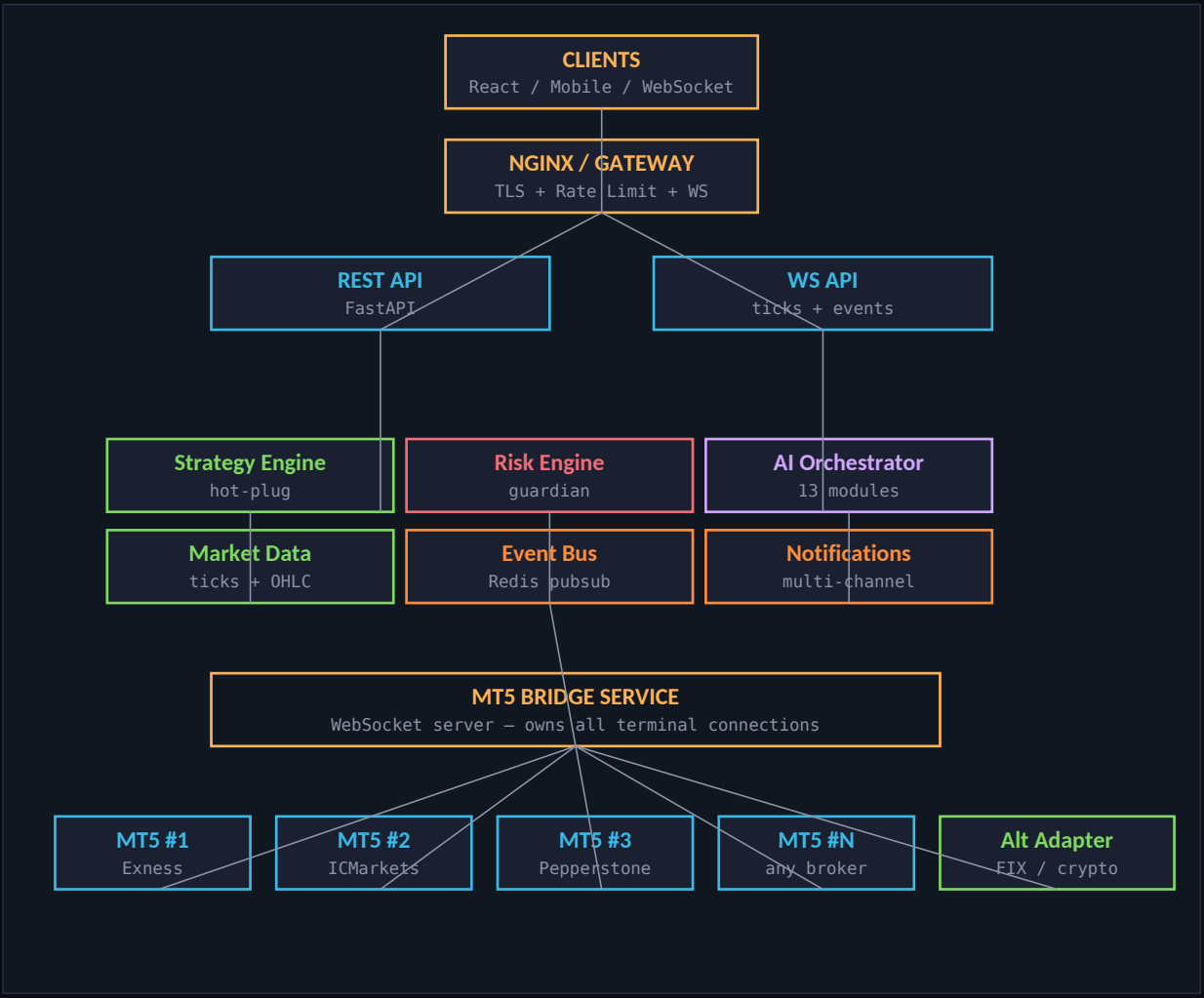

Figure 3.1 — High-level system topology. Arrows show primary data flow; the Bridge Service is the only component that talks to MT5 terminals.

3.1 Layered Architecture (Clean + DDD + Hexagonal)

The codebase is organized into four concentric layers. Each layer may only depend on layers inward of it. The domain layer is the innermost; it has no dependencies on frameworks, databases, or external services. The application layer orchestrates use cases by loading aggregates from repositories, invoking domain methods, and persisting the results. The infrastructure layer provides concrete implementations of repository protocols, the MT5 Bridge client, Redis adapters, and Celery workers. The API layer is a thin translation surface that converts HTTP/WebSocket requests into application-layer commands and serializes the responses.

Layer	Responsibility	Modules
domain	Pure business logic — aggregates, value objects, events	Trading, Risk, Strategy, MarketData, AI, Identity
application	Use-case handlers (CQRS commands + queries)	PlaceOrder, CancelOrder, RunBacktest, SyncAccount
infrastructure	Adapters — DB, Redis, Bridge, Celery, LLM APIs	SQLAlchemy repos, BridgeClient, Celery tasks
api	HTTP/WebSocket translation surface	REST routers, WebSocket routers, auth middleware

bridge	WebSocket server for MT5 terminals (separate process)	BridgeSession, handlers, registry, command queue
core	Cross-cutting concerns shared by all layers	config, logging, security, telemetry, exceptions

The hexagonal (ports-and-adapters) flavor of this architecture is most visible in the execution subsystem: the domain defines the `ExecutionAdapter` protocol as a port, and the infrastructure layer provides adapters for MT5 (via the Bridge), paper broker (for backtesting), and future venues. The application layer never knows which adapter is wired in; it just calls `adapter.place_order(req)`.

3.2 Component Map

The table below lists every top-level module in the platform, its responsibility, and the primary technologies it depends on. This map is the canonical reference for code navigation — when a new engineer asks 'where does X live?', this is the answer.

Module	Responsibility	Deps
core	Config, logging (structlog), security (JWT, bcrypt, API keys), telemetry (Prometheus + OTel), exception hierarchy	Pydantic Settings, structlog, PyJWT, passlib
db	SQLAlchemy 2.0 async ORM models, session factory, Alembic migrations	SQLAlchemy, asyncpg, Alembic
domain	DDD aggregates (Order, Position), value objects (Money, Price, Quantity), domain events	Pure Python — no I/O
application	CQRS command/query handlers — PlaceOrder, CancelOrder, RunBacktest, etc.	Domain + repositories
infrastructure.mt5_bridge	Bridge protocol, terminal registry, command queue, bridge client	Pydantic, asyncio, websockets
infrastructure.execution	ExecutionAdapter protocol (port) — MT5, paper, FIX, crypto adapters	abc
infrastructure.celery_app	Background workers — tick persistence, execution persistence, backtests, notifications	Celery, Redis
api.v1	REST routers — auth, terminals, strategies, orders, AI, risk, analytics, admin, market-data	FastAPI, Pydantic
api.ws	WebSocket routers — live ticks, terminal events fanout	FastAPI WebSocket, asyncio
bridge	Bridge Service process — owns all MT5 terminal connections	websockets, asyncio
strategies	Strategy SDK + built-in strategies (EMA cross, RSI reversion, SMC)	Pure Python
ai	AI module SDK + Trend/Risk/Pattern AIs + orchestrator + LLM assistant	scikit-learn, XGBoost, ONNX, httpx
risk	Risk engine + rule pack (KillSwitch, MaxDailyLoss, MaxDrawdown, ...)	Pure Python
market_data	Tick ingestion, OHLC aggregation, multi-TF cache, replay engine	asyncio, Redis
events	Event bus — Redis pubsub in prod, in-process in tests	redis.asyncio
notifications	Multi-channel notifications — email, Telegram, Discord, webhook	aiosmtplib, httpx
observability	Health checks, readiness probes, structured log forwarding	prometheus-client
plugins	Plugin loader for user-supplied strategies, AIs, brokers, channels	importlib.metadata

3.3 Data Flow — From Tick to Trade

The canonical end-to-end flow is: an MT5 terminal streams a tick → the Bridge Service publishes it on the `atlas.ticks` Redis topic → the market-data engine aggregates it into OHLC bars across all configured timeframes → when a bar closes, the

strategy engine evaluates every active strategy subscribed to that symbol/timeframe → strategies emit signals → the risk engine validates each signal against all registered rules → if approved, the application layer issues a `place_order` command to the Bridge → the Bridge dispatches it over WebSocket to the terminal's EA → the EA executes the trade and returns an execution report → the Bridge publishes the report → the application layer updates the Order aggregate and persists it → the WebSocket API fans the update out to subscribed dashboards.

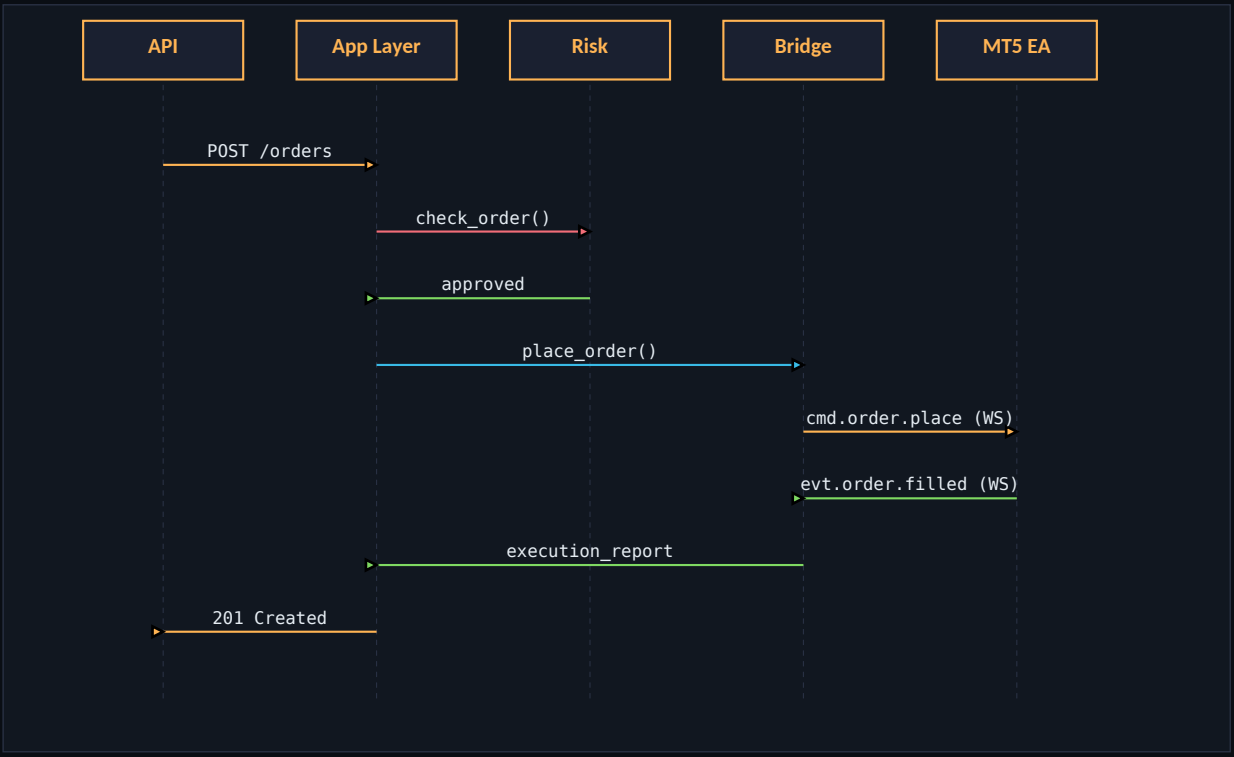

Figure 3.2 — Order placement sequence: API → App Layer → Risk → Bridge → MT5 EA, with execution report flowing back through the same path.

4. MT5 Bridge Service (Deep Dive)

The Bridge Service is the most important component in the platform. It is the single chokepoint through which all communication with MT5 terminals flows. By centralizing this responsibility, the platform gains three properties that would otherwise be impossible: (1) the backend never has to know how to talk to MT5, only how to talk to the Bridge; (2) the Bridge can be scaled, sharded, and made highly available independently of the rest of the backend; (3) the Bridge can be replaced entirely (e.g. with a native gRPC client for a future broker SDK) without touching the application layer.

4.1 Why a Bridge? Wine Discipline

MetaTrader 5 is a Windows application. On Linux it runs under Wine, which is itself a complex compatibility layer. Directly integrating the MT5 Python package (`MetaTrader5`) into the FastAPI backend would mean: (a) the backend process would have to be co-located with the MT5 terminal on the same host, killing horizontal scalability; (b) every MT5 crash would take down the API; (c) multi-terminal support would require spinning up multiple Python interpreters inside the same process, with all the GIL contention that implies; and (d) the backend would inherit every Wine quirk as a first-class problem.

The Bridge eliminates all of these issues. The backend process is a pure async Python service that talks to the Bridge over a clean JSON protocol carried by WebSocket. The Bridge is a separate process that owns the WebSocket server terminals connect to. The MT5 terminal runs under Wine on the host, and the MQL5 EA inside it opens a WebSocket connection to the Bridge. Wine is treated as an OS dependency — the backend never manages it, never inspects it, never restarts it. systemd handles Wine process supervision; the Bridge handles terminal lifecycle.

⚠ **Wine Rule (enforced).** No code in `platform/` may import `wine`, `subprocess` with `wine` as target, or `MetaTrader5`. All MT5 access goes through `platform.infrastructure.mt5_bridge.client.BridgeClient`. CI enforces this via a grep gate in the test suite.

4.2 Bridge Protocol v1

The protocol is a bidirectional JSON message envelope carried over a single WebSocket connection per terminal. Every message has the same five top-level fields: `v` (protocol version), `t` (message type — `cmd.*` for server-to-terminal commands, `evt.*` for terminal-to-server events), `id` (correlation UUID for matching request and reply), `ts` (ISO-8601 timestamp), `terminal_id` (the terminal's externally-known identifier), and `payload` (a type-specific dict). Responses echo the original command's `id` in a `reply_to` field so the command queue can resolve the awaiting future.

The protocol is intentionally transport-agnostic. Today it is carried by WebSocket; tomorrow it could be gRPC, AMQP, or a FIX-like binary frame. Only the framing layer changes — the message envelope does not. This makes the Bridge future-proof against transport evolution.

Message envelope (Pydantic)

```
class BridgeMessage(BaseModel):
    v: int = 1                      # protocol version
    t: str                           # "cmd.order.place" | "evt.tick" | ...
    id: str = uuid4()                # correlation id
    ts: datetime = utcnow()
    terminal_id: str | None = None
    reply_to: str | None = None      # for responses
    payload: dict = {}
```

Command types (server → terminal)

Command	Purpose
cmd.ping	Liveness probe
cmd.order.place	Place a market/limit/stop order
cmd.order.modify	Modify SL/TP on a pending order
cmd.order.cancel	Cancel a pending order

cmd.position.close	Close (or partially close) an open position
cmd.position.modify	Modify SL/TP on an open position
cmd.position.sync	Request full snapshot of open positions
cmd.order.sync	Request full snapshot of pending orders
cmd.account.sync	Request account balance/equity/margin snapshot
cmd.ticks.subscribe	Subscribe to ticks for a set of symbols
cmd.ticks.unsubscribe	Stop streaming ticks for given symbols
cmd.symbols.subscribe	Add symbols to terminal's symbol watchlist
cmd.history.get	Get historical OHLC for a range
cmd.ea.restart	Restart the EA (rarely used; for recovery)
cmd.account.flatten	Close ALL positions and cancel ALL orders (emergency)

Event types (terminal → server)

Event	Purpose
evt.register	Terminal handshake — identifies itself + auth token
evt.heartbeat	Periodic liveness (default every 10s)
evt.tick	Streaming tick (bid/ask/last/volume/ts)
evt.bar	Closed OHLC bar (optional — server can aggregate from ticks)
evt.order.accepted	Broker acknowledged pending order
evt.order.rejected	Broker rejected the order (with reason)
evt.order.partial	Partial fill
evt.order.filled	Full fill (with broker_execution_id, avg_price)
evt.order.cancelled	Cancellation confirmed
evt.position.opened	New position opened
evt.position.modified	Position SL/TP changed
evt.position.closed	Position closed (with realized PnL)
evt.account.update	Balance/equity/margin changed
evt.symbol.info	Symbol metadata (digits, contract size, vol limits)
evt.error	Terminal-side error report
evt.log	Terminal-side log line (forwarded to structured logger)

4.3 Terminal Registry & Heartbeats

The Bridge Service maintains an in-memory `TerminalRegistry` that tracks every connected terminal. Each entry holds the terminal's identity, its registered symbols, its capabilities, a reference to the live `BridgeSession`, and the timestamp of the last heartbeat. A background watcher task iterates every 5 seconds and marks terminals as `degraded` if no heartbeat has arrived within the configured timeout (default 30s), then `offline` if no heartbeat arrives within 2× the timeout. Offline terminals are evicted — their session is closed and the registry entry removed — so that subsequent commands targeting them return a clean `TerminalOffline` error rather than blocking on a dead connection.

When a terminal reconnects (e.g. after a Wine crash), it re-issues `evt.register`. The registry detects that an entry already exists for the same `terminal_id`, politely closes the old session with code 4001 ('Replaced by new connection'), and replaces

the entry. This handles the common case where a terminal reconnects before the old connection's TCP teardown has propagated.

Registration message (sent by the EA at startup)

```
{
  "v": 1,
  "t": "evt.register",
  "terminal_id": "mt5-exness-01",
  "payload": {
    "terminal_id": "mt5-exness-01",
    "broker": "Exness",
    "account": 123456789,
    "version": "5.0.0.2380",
    "symbols": ["XAUUSD","XTIUSD","EURUSD","GBPUSD"],
    "auth_token": "***",
    "capabilities": {
      "market": true, "limit": true, "stop": true,
      "close_partial": true, "modify_sl_tp": true
    }
  }
}
```

4.4 Command Queue & Acknowledgement

Every command sent to a terminal gets a future. The Bridge Service's `CommandQueue` tracks pending commands per terminal in a dict-of-dicts keyed by `terminal_id → command_id → PendingCommand`. When the terminal returns an execution report that echoes the original command's `id` in its `reply_to` field, the queue resolves the future with the reply message. If no reply arrives within the command's timeout (default 10s for orders, 30s for sync operations), the queue fails the future with a `CommandTimeout` exception, which the application layer maps to an HTTP 504.

This pattern is critical for keeping the application layer simple. The `PlaceOrder` command handler is a synchronous-looking async function that calls `await bridge.place_order(...)` and gets back the execution report. Behind the scenes, the Bridge sent a command, awaited an event, and matched them via correlation id — but the caller doesn't have to know that. The same handler would work unchanged if the Bridge swapped WebSocket for gRPC tomorrow.

When a terminal goes offline unexpectedly, the `fail_all` method fails every pending command for that terminal with a `RuntimeError('terminal disconnected')`. This unblocks any application-layer coroutines that were awaiting replies, allowing them to roll back pending state and return a clean error to the API caller.

4.5 Reconnection & Error Recovery

Reconnection is handled at three layers, each with a different scope and timeout. Layer 1: the WebSocket transport. The `websockets` library sends protocol-level pings every 20 seconds and expects a pong within 10 seconds; if either fails, the connection is dropped and the Bridge's per-connection handler exits. The registry eviction logic kicks in via the heartbeat watcher. Layer 2: the EA's reconnect loop. If the WebSocket drops, the EA's `OnTimer` detects the closed socket, sleeps for `InpReconnectMs` (default 3000ms), and reconnects. Once reconnected, it re-registers, which causes the Bridge to replace the old session. Layer 3: systemd on the host. If Wine itself crashes, systemd restarts the Wine unit, which restarts MT5, which reloads the EA, which reconnects to the Bridge.

The combination of these three layers gives us end-to-end recovery without manual intervention. The platform can survive: a transient network blip (recovered in seconds by the EA's reconnect loop), an MT5 freeze (recovered by Wine process watchdog), or a full host crash (recovered by systemd boot ordering + EA auto-attach). The backend, meanwhile, is quiescent: pending commands fail fast, the affected terminal is marked offline, and the system continues serving other terminals normally.

4.6 Multi-Terminal Support

The Bridge Service is designed from the ground up to handle an arbitrary number of terminals. There is no hard-coded terminal list; terminals self-register at startup. The MQL5 EA's `InpTerminalId` input parameter gives each terminal a unique human-readable identifier (e.g. `mt5-exness-01`, `mt5-icmarkets-01`), and the Bridge routes commands based on that

identifier. The application layer's `BridgeClient` takes a `terminal_id` argument on every call — there is no global 'current terminal' state.

When a strategy or use case needs to choose a terminal, it queries the registry via `select_for_symbol(symbol, preferred_broker=...)`. The current implementation is crude — it picks the first online terminal that has the symbol — but the registry interface is stable, so the implementation can be upgraded to load-aware or latency-aware routing without changing call sites. Future versions will factor in terminal latency, account free margin, and broker-specific cost models.

4.7 Bridge Sharding & High Availability

A single Bridge process can comfortably handle a few hundred terminals — WebSocket connections are cheap, and the per-message processing is dominated by JSON serialization. For larger fleets, the Bridge can be sharded: a Redis-backed map of `terminal_id → bridge_node_id` is consulted at routing time, and a sticky-session load balancer (Nginx with `ip_hash` or HAProxy with `hash terminal_id`) ensures that a given terminal always lands on the same Bridge node. When a Bridge node fails, its terminals reconnect through the load balancer to a surviving node, which re-registers them and rebuilds its in-memory registry from the registration events. State loss is zero — the registry is a cache of terminal liveness, not authoritative state.

⚠ **HA recipe for production.** Run N Bridge nodes behind HAProxy with `balance source`. Run N FastAPI API nodes behind Nginx with `least_conn`. Run PostgreSQL with synchronous streaming replication + Patroni for automatic failover. Run Redis with sentinel. Total hardware: 3 small VMs for stateless services, 2 larger VMs for PostgreSQL, 3 small VMs for Redis sentinel.

5. Domain Model (DDD)

The domain layer contains six bounded contexts, each with its own aggregate roots, entities, value objects, and domain events. The contexts are: Identity (User, Organization, APIKey), Trading (Order, Execution, Position, Trade), Strategy (Strategy, Signal), Risk (RiskEvent, RiskRule), MarketData (Symbol, Tick, Candle), and AI (AIResult, Prediction). Each context has a corresponding module under `platform/domain` and a set of repository protocols that the infrastructure layer implements.

5.1 Trading aggregate

The Trading aggregate is the heart of the system. `Order` is an aggregate root with strict state transitions: `PENDING` → `SUBMITTED` → `PARTIAL` → `FILLED`, or → `CANCELLED`, or → `REJECTED`. Each transition is a method on the aggregate that validates the current state and raises a `DomainError` if the transition is illegal. The `apply_fill` method is the most interesting: it updates the weighted average fill price, emits an `OrderFilled` domain event for each individual fill, and transitions to `FILLED` only when the cumulative filled volume equals the requested volume.

Order state machine (Pydantic + dataclass)

```
class Order(AggregateRoot):
    status: OrderStatus = OrderStatus.PENDING
    filled_volume: float = 0.0
    avg_fill_price: float | None = None

    def mark_submitted(self) -> None:
        if self.status != OrderStatus.PENDING:
            raise DomainError(...)
        self.status = OrderStatus.SUBMITTED

    def apply_fill(self, filled_volume, fill_price) -> list[DomainEvent]:
        if self.status in (FILLED, CANCELLED, REJECTED):
            raise DomainError(...)
        new_total = self.filled_volume + filled_volume
        if new_total > self.volume.volume:
            raise DomainError("Fill exceeds order volume")
        # update weighted average
        self.avg_fill_price = (prev*old + filled*new) / new_total
        self.filled_volume = new_total
        if new_total == self.volume.volume:
            self.status = OrderStatus.FILLED
        else:
            self.status = OrderStatus.PARTIAL
        return [OrderFilled(order_id=self.id, ...)]
```

5.2 Value objects

Value objects are immutable, structurally comparable, and self-validating. `Money` rejects negative amounts and refuses to add across currencies. `Price` rejects zero and negative values, and exposes a `point` property derived from `symbol_digits`. `Quantity` validates that the volume is aligned to the symbol's minimum step. `Timeframe` validates against the supported set and exposes a `seconds` property. These small objects eliminate an entire class of bugs (negative prices, mismatched currencies, unaligned volumes) by failing fast at construction time.

5.3 Domain events

Aggregate roots record domain events as they transition state. The `collect_events()` method drains the internal queue and returns the events to the caller (typically the application layer), which publishes them on the event bus. This pattern keeps the domain layer pure — it doesn't know about Redis or Celery — while still enabling event-driven side effects (notifications, audit, analytics) in the infrastructure layer.

Event	Emitter	Meaning

OrderPlaced	Order	Order entered PENDING state
OrderFilled	Order	Partial or full fill received from broker
PositionOpened	Position	New position created by a fill
PositionClosed	Position	Position closed — carries realized PnL
TerminalRegistered	Terminal	Terminal completed handshake
TerminalWentOffline	Terminal	Terminal missed heartbeat threshold
RiskLimitBreached	RiskEvent	A risk rule rejected an order

6. Strategy Engine

The strategy engine is the decision-maker. It hosts hot-pluggable strategies, each implementing a small interface (`on_bar`, `on_tick`, `on_start`, `on_stop`) and producing `Signal` objects that the application layer feeds into the risk engine and execution pipeline. Strategies are pure functions of market data plus their own state; they have no database access, no HTTP access, no side effects beyond emitting signals. This makes them trivially testable and safe to hot-swap.

6.1 Strategy SDK

Every strategy subclasses `Strategy` and is registered with the `StrategyRegistry` via the `@strategy` decorator. The registry holds class objects keyed by name; instances are created per-org with the stored config merged over the class's `default_config`. The `StrategyContext` passed to each call carries the org ID, terminal ID, strategy ID, merged config, and a per-strategy scratchpad dict for stateful computations.

Minimal strategy (real example from the codebase)

```python
@strategy
class EMACrossStrategy(Strategy):
    name = "ema_cross"
    default_config = {"fast_period": 9, "slow_period": 21}

    def __init__(self, *, fast_period=9, slow_period=21):
        self.fast_period = fast_period
        self.slow_period = slow_period
        self._cur_fast = 0.0
        self._cur_slow = 0.0

    async def on_bar(self, bar: Bar, ctx: StrategyContext) -> Signal | None:
        if not bar.is_closed:
            return None
        self._cur_fast = ema(self._cur_fast, bar.close, self.fast_period)
        self._cur_slow = ema(self._cur_slow, bar.close, self.slow_period)
        # detect cross ...
        return Signal(symbol=bar.symbol, side="buy", strength=strength)
```

6.2 Built-in strategies

Name	Logic	Best For
ema_cross	EMA fast/slow crossover	Trend-following, swing
rsi_reversion	RSI overbought/oversold reversion	Mean-reversion, range
smc_order_blocks	Smart Money Concepts — order blocks + FVG	Discretionary, intraday
macd_divergence	MACD histogram divergence	Trend exhaustion
liquidity_sweep	Liquidity grab + reversal candle	ICT-style
breakout	Donchian / Bollinger breakout	Trend continuation
grid	Symmetric grid around mid	Range, high freq
news_overlay	Suppress signals around high-impact news	Risk filter

Beyond the built-ins, the platform supports a custom Python Strategy SDK that lets users upload their own strategies as Python source files. Uploaded strategies are loaded into a sandboxed namespace, validated against the Strategy protocol, and registered with the registry. The sandbox restricts imports to a curated whitelist (numpy, pandas, talib, sklearn) and enforces per-strategy CPU/memory budgets via asyncio timeouts and resource limits.

7. AI Engine

The AI engine is a heterogeneous mix of specialized modules, each addressing one analytical dimension. Modules are independent: each one trains its own models, has its own versioning, and emits predictions in a uniform format. The orchestrator fan-ins all module outputs into a composite score that the strategy engine can consume as a tie-breaking signal or as an override under specific market regimes.

7.1 Module catalog

Module	Responsibility	Model Type
trend	EMA slope + ADX regime classifier	ONNX (trained offline)
pattern	Candlestick pattern CNN classifier	PyTorch → ONNX
sentiment	News + social NLP sentiment (per symbol)	Transformer (HuggingFace)
economic_calendar	Impact-weighted event forecast	XGBoost regression
portfolio	Cross-asset correlation + rebalance advice	scikit-learn
risk	Volatility regime + suggested position sizing	GARCH + heuristic
volatility	ATR / implied vol regime classifier	GARCH
trade_quality	Post-trade quality scoring (slippage, timing)	XGBoost
position_size	Kelly-criterion with drawdown cap	Closed-form
trade_journal	LLM-based trade journaling + retrospective	LLM
strategy_generator	LLM that emits candidate strategy code	LLM + code sandbox
llm_assistant	Conversational trading copilot	LLM (OpenAI / vLLM / Ollama)
anomaly_detection	Unusual tick / volume / spread detection	Isolation Forest

7.2 Orchestrator

The orchestrator runs every module on the current context, collects predictions, and computes a weighted composite score in the range [-1.0, +1.0]. Weights are per-module and confidence-scaled — a high-confidence Trend AI prediction gets more weight than a low-confidence Sentiment AI one. The composite score is exposed via the `/api/v1/ai/analyze` endpoint and is also published on the `atlas.ai_results` topic so strategies can subscribe to it.

7.3 Hybrid LLM stack

ATLAS uses a hybrid LLM strategy. All inference-heavy AIs (trend, pattern, risk, anomaly) run local models — scikit-learn, XGBoost, ONNX runtime — for latency, cost, and privacy. The LLM-powered modules (trade journal, strategy generator, conversational assistant) call external LLM APIs (OpenAI, Anthropic) or self-hosted vLLM/Ollama endpoints depending on configuration. The `LLM_PROVIDER` setting (`openai | anthropic | vllm | ollama | none`) picks the backend; the `llm_assistant` module transparently adapts. Switching providers is a config change, not a code change.

8. Risk Engine

The risk engine is the guardian. Every order passes through `risk.check_order(...)` before it leaves the system. If any rule rejects, the order never reaches the Bridge. The engine is pluggable — adding a rule is a single `register()` call — and every rejection is published on the `atlas.risk_events` topic, where it gets persisted to the `risk_events` table and fanned out to the WebSocket API for real-time dashboard updates.

8.1 Rule catalog

Rule	Purpose	Severity
kill_switch	Manual / automatic hard stop. Rejects ALL orders.	Critical
max_daily_loss	Rejects new orders if org hit its daily loss limit (USD).	Critical
max_weekly_loss	Same as above, weekly window.	Critical
max_drawdown	Rejects if account drawdown exceeds threshold (e.g. 20%).	Critical
position_limit	Rejects if total open positions exceed configured max.	Warning
max_exposure	Rejects if aggregate notional exposure exceeds cap.	Warning
correlation_risk	Rejects if new position would push portfolio correlation > threshold.	Warning
sector_exposure	Rejects if sector exposure exceeds cap (e.g. >50% in metals).	Warning
spread_protection	Rejects if bid-ask spread > threshold (low-liquidity guard).	Warning
news_lock	Blocks new orders within ±N minutes of high-impact news.	Warning
volatility_lock	Blocks new orders when ATR% > threshold.	Warning
kelly_sizing	Suggests position size based on win rate + payoff ratio.	Info

8.2 Kill switch

The kill switch is the highest-authority rule. When engaged, every `check_order` call raises `RiskLimitBreached` immediately — no other rule runs. It can be engaged manually via the `/api/v1/risk/kill-switch/engage` endpoint (admin or trader role required) or automatically by other rules (e.g. max_drawdown engages the kill switch when triggered). Release requires admin role, preventing accidental de-escalation by traders in a panic.

8.3 Position sizing

Beyond blocking, the risk engine also proposes position sizes. The default `kelly_sizing` rule computes the Kelly fraction from the strategy's historical win rate and payoff ratio, applies a configurable drawdown cap (typically 0.25× Kelly to reduce volatility), and returns the suggested volume. The application layer passes this suggestion to the strategy, which may use it as-is, cap it, or ignore it. This separation keeps sizing policy in one place while letting strategies retain final say.

9. Market Data Engine

The market data engine ingests ticks, aggregates OHLC bars across all configured timeframes, and publishes closed bars back on the event bus for strategies and AI modules to consume. It is purely event-driven: it subscribes to the `atlas.ticks` topic at startup and maintains in-memory `TimeframeBucket` objects per (symbol, timeframe) pair. When a tick arrives, every bucket ingests it; when a bucket crosses a timeframe boundary, the previous bar is marked closed and published.

9.1 Storage tiers

Tier	Location	Notes	Capacity
Hot	In-memory current-bar cache	Sub-ms reads, no persistence	Tens of MB
Warm	PostgreSQL candles table	Indexed by (symbol, timeframe, ts)	Millions of rows
Cold	PostgreSQL ticks table (partitioned by month)	Append-only, batch writes	Billions of rows
Archive	S3 / object storage parquet dumps	Quarterly snapshots	Unbounded

For high-throughput production deployments, the cold tier should be migrated to TimescaleDB's hypertable extension. The schema is already compatible — the `ticks` table just needs `SELECT create_hypertable('ticks', 'ts')` run once after migration. TimescaleDB gives automatic time-based partitioning, compression of old data, and continuous aggregates for common OHLC queries, often delivering 10-100× query speedups over vanilla PostgreSQL for tick-range scans.

9.2 Replay engine

For backtesting and post-mortem analysis, the engine can replay historical ticks at real-time speed or as fast as possible. The replay reads from the cold tier and publishes synthetic `atlas.ticks` events with a `replay: true` flag. Strategies and AIs that subscribe to the topic process them identically to live ticks — the same code path runs in backtest and in production, eliminating a whole class of backtest/live divergence bugs.

10. Database Schema (PostgreSQL)

The schema is normalized 3NF for transactional tables, denormalized for hot read paths. Every user-owned table carries `org_id` for multi-tenant isolation. Time-series tables (`ticks`, `candles`) are partitioned by time in production; the Alembic migration creates them as plain tables and a follow-up migration converts them to TimescaleDB hypertables when the extension is available.

10.1 Table inventory

Table	Purpose	Key Columns
organizations	Tenancy root — one row per customer org	id, slug, plan, settings
users	User accounts within an org	email, role, password_hash
api_keys	Long-lived API credentials	key_hash, scopes, expires_at
brokers	Broker integrations per org	code, adapter_kind, credentials
terminals	Registered MT5 terminals	terminal_id, broker_account, status
accounts	Trading accounts (one per broker login)	balance, equity, margin
strategies	Strategy definitions (config + version)	kind, config, is_active
signals	Strategy-emitted signals (input to orders)	symbol, side, strength
orders	Order lifecycle — pending → filled/cancelled	client_order_id, status
executions	Individual fills (orders may have many)	volume, price, commission
positions	Open positions (mark-to-market)	unrealized_pnl, current_price
trades	Closed positions — historical record	pnl, pips, duration
symbols	Symbol metadata (digits, contract size, vol limits)	name, digits
ticks	Raw tick storage (partitioned)	bid, ask, ts
candles	OHLC cache per (symbol, timeframe, ts)	open, high, low, close
ai_results	AI module predictions + confidence	module, prediction, confidence
risk_events	Risk rule rejections / warnings	rule, severity, action
audit_logs	Append-only audit trail	actor, action, payload
notifications	Outbound notification queue	channel, status, sent_at
backtests	Backtest runs + results	config, results, sharpe

10.2 Indexing strategy

Indexes are chosen by query pattern, not by column cardinality in isolation. The most common access patterns are: list X by org_id (every dashboard query), filter by status (active orders, open positions), time-range scans on ticks/candles, and point lookups by client_order_id / broker_order_id. Composite indexes cover the first three: `(org_id, status)` on terminals and orders, `(org_id, terminal_id, status)` on positions, `(symbol, ts)` on ticks. The unique constraint `uq_candles_term_sym_tf_ts` on candles doubles as a covering index for OHLC reads.

11. API Specification

The API surface is split into a REST API (CRUD + commands) and a WebSocket API (real-time streaming). Both live behind the same Nginx edge, share the same auth (JWT or API key), and produce/consume the same Pydantic models. Full OpenAPI 3.1 spec is auto-generated by FastAPI and available at `/docs` in development, `/openapi.json` in production.

11.1 REST API (v1)

Method	Path	Purpose
POST	/api/v1/auth/login	Email + password → JWT pair
POST	/api/v1/auth/refresh	Refresh token → new JWT pair
POST	/api/v1/auth/api-keys	Issue new API key (shown once)
GET	/api/v1/terminals	List terminals for org
GET	/api/v1/terminals/{terminal_id}	Terminal detail + live status
POST	/api/v1/terminals/{terminal_id}/sync-positions	Force position sync
POST	/api/v1/terminals/{terminal_id}/sync-account	Force account sync
GET	/api/v1/strategies	List configured strategies
POST	/api/v1/strategies	Create new strategy
GET	/api/v1/strategies/available	List SDK-known strategy kinds
POST	/api/v1/strategies/{id}/activate	Activate strategy
POST	/api/v1/strategies/{id}/deactivate	Deactivate strategy
POST	/api/v1/orders	Place new order → execution result
GET	/api/v1/orders	List orders (filter by status)
GET	/api/v1/orders/{id}	Order detail
GET	/api/v1/market-data/candles/{symbol}	Historical OHLC
GET	/api/v1/market-data/symbols	Symbol metadata catalog
POST	/api/v1/ai/analyze	Run AI orchestrator on (symbol, features)
POST	/api/v1/ai/assistant/chat	LLM assistant free-form chat
GET	/api/v1/risk/kill-switch	Check kill switch state
POST	/api/v1/risk/kill-switch/engage	Engage kill switch (trader+)
POST	/api/v1/risk/kill-switch/release	Release kill switch (admin)
GET	/api/v1/analytics/performance	P&L summary (last N days)
GET	/api/v1/analytics/positions/open	Currently open positions
GET	/api/v1/admin/status	System status (admin only)

11.2 WebSocket API

WebSocket endpoints require authentication via a `?token=<jwt>` query parameter (browsers cannot easily set headers on WebSocket handshakes). After accepting the connection, the client sends a subscription message; the server then pushes matching events as JSON. A 30-second idle timer sends a ping to keep the connection alive through proxies.

Path	Purpose	Subscription
/ws/ticks	Live tick stream	{"symbols":["XAUUSD","EURUSD"]}
/ws/terminal-events	Terminal lifecycle + executions + risk events	(no body)
/ws/orders	Per-org order state changes	(no body)
/ws/positions	Position updates (open, modify, close)	(no body)
/ws/ai-insights	AI module predictions in real time	{"modules":["trend","risk"]}

12. SDKs

ATLAS ships three SDKs that define extension points: the Strategy SDK (Python), the AI Module SDK (Python), and the MQL5 Expert Advisor SDK (MQL5). Each SDK has a base class or contract, a registration mechanism, and a curated set of built-in implementations demonstrating usage. New functionality is added by implementing the SDK contract and registering the implementation — no core code changes required.

12.1 Strategy SDK

Subclass `Strategy`, decorate with `@strategy`, implement `on_bar` (and optionally `on_tick`, `on_start`, `on_stop`). Return a `Signal` or `None`. The platform handles everything else: scheduling, dispatch, subscription management, persistence, risk integration, and execution. See §6.1 for a complete example.

12.2 AI Module SDK

Subclass `AIModule`, implement `analyze(ctx) → AIPrediction`, register with the orchestrator. The orchestrator runs every registered module on each context, collects predictions, and computes a weighted composite score. Modules are independent: one failing module does not poison the orchestrator. See §7 for the catalog and the orchestrator implementation.

12.3 MQL5 Expert Advisor SDK

The `BridgeEA.mq5` file is the entire SDK. It is a single Expert Advisor that runs inside MetaTrader 5, opens a WebSocket connection to the Bridge Service, registers itself, streams ticks and execution reports, and dispatches commands (place/cancel/modify order, close position, sync). The EA is configurable via input parameters — bridge URL, terminal ID, broker name, auth token, symbol list, heartbeat interval, reconnect delay — and is designed to be attached to a single chart per MT5 instance. Multiple MT5 instances on the same host each run their own copy of the EA with different terminal IDs.

EA input parameters

Parameter	Default	Purpose
InpBridgeUrl	ws://127.0.0.1:9000	Bridge Service WebSocket URL
InpTerminalId	mt5-exness-01	Unique terminal identifier
InpBroker	Exness	Human-readable broker name
InpAuthToken	***	Must match BRIDGE_AUTH_TOKEN on backend
InpSymbolsCSV	XAUUSD,XTIUSD,EURUSD	Symbols to stream ticks for
InpHeartbeatSeconds	10	Heartbeat frequency
InpReconnectMs	3000	Reconnect delay on disconnect
InpMagic	770000	Magic number for EA-placed orders

13. Plugin Architecture

Five plugin extension points exist in the platform: strategies, AI modules, risk rules, execution adapters, and notification channels. Each follows the same pattern: a base class or protocol defines the contract, a registry holds implementations keyed by name, and a loader discovers plugins via Python's `importlib.metadata` entry points (or, for user-uploaded plugins, via dynamic import from a sandboxed namespace).

Extension Point	Base Class	Registration	Built-in Location
strategies	Strategy	@strategy decorator	platform.strategies.builtin
ai_modules	AIModule	AIOrchestrator.register()	platform.ai.modules
risk_rules	RiskRule	RiskEngine.register()	platform.risk.rules
execution_adapters	ExecutionAdapter	AdapterRegistry.register()	platform.infrastructure.execution
notification_channels	NotificationChannel	ChannelRegistry.register()	platform.notifications.channels

13.1 Adding a new broker (non-MT5)

Suppose we want to add Binance as an execution venue. The work is: (1) implement `BinanceAdapter(ExecutionAdapter)` in a new file under `platform/infrastructure execution/`; (2) register it via `@execution_adapter`; (3) write a thin 'bridge' (or rather, a connector) that maintains the WebSocket connection to Binance and translates between Binance's JSON format and our internal BridgeMessage format. The application layer, the strategy engine, the risk engine, the AI orchestrator — none of them change. New orders can be routed to Binance by passing `terminal_id="binance-spot-01"` to `bridge.place_order`.

14. Event-Driven Architecture

The platform uses an event-driven backbone for the hot path: ticks, execution reports, position updates, account updates, signals, AI results, risk events, and terminal lifecycle events all flow through a Redis pubsub layer. The event bus is a two-tier affair: in-process handlers are invoked synchronously (low latency for the hot path), and cross-process handlers receive the same events via Redis pubsub. This gives us the latency of in-process calls for same-process subscribers (e.g. market-data engine subscribing to ticks) and the decoupling of pubsub for cross-process subscribers (e.g. Celery workers persisting ticks to PostgreSQL).

14.1 Topic catalog

Topic	Payload	Notes
atlas.ticks	TickPayload	Hot — every tick from every terminal
atlas.execution_reports	ExecutionReportPayload	Order state changes
atlas.position_updates	PositionUpdatePayload	Position open/modify/close
atlas.account_updates	AccountUpdatePayload	Balance/equity/margin changes
atlas.signals	Signal	Strategy-emitted signals
atlas.orders	OrderEvent	Order lifecycle
atlas.risk_events	RiskEvent	Risk rule rejections / warnings
atlas.ai_results	AIPrediction	AI module predictions
atlas.terminal_events	TerminalEvent	Terminal register/offline/recover
atlas.notifications	Notification	Outbound notification queue
atlas.audit	AuditEntry	Append-only audit trail

14.2 Delivery guarantees

Redis pubsub is at-most-once: if a subscriber is slow or disconnected, it misses messages. For most topics this is fine — a missed tick is irrelevant; the next one arrives in milliseconds. For events that must not be lost (executions, risk events, audit entries), the publisher also writes to PostgreSQL in the same transaction as the state change. Consumers that need durability subscribe to the topic for notifications and read from PostgreSQL for the authoritative record.

For work that needs at-least-once delivery with retries (e.g. sending an email notification when a risk rule triggers), Celery tasks are dispatched from event handlers. The Celery task retries with exponential backoff; idempotency keys prevent duplicate side effects if a task is redelivered.

15. Security Architecture

Security is layered: transport, authentication, authorization, input validation, secrets management, audit logging, and rate limiting. Each layer is independently testable and independently configurable.

15.1 Authentication

Two authentication methods: JWT (for human users via the web UI) and API keys (for automated clients). JWTs are HS256-signed with the platform secret, have a 15-minute access TTL and a 30-day refresh TTL, and carry the user's `org_id`, `user_id`, role, and scopes in claims. API keys are SHA-256-hashed with bcrypt at rest; the raw key is shown once at creation time and never persisted. Both methods resolve to a `CurrentUser` dataclass that downstream code depends on.

15.2 Authorization (RBAC)

Role	Capabilities	Scope
admin	Full org access including user management + kill switch release	All endpoints
trader	Place/cancel orders, manage strategies, engage kill switch	Most endpoints except admin
viewer	Read-only access to dashboards and analytics	GET endpoints only
bot	API-key-only — restricted to scoped endpoints	Per API key scopes

15.3 Secrets management

Secrets (DB passwords, JWT secret, bridge auth token, LLM API keys, broker credentials) are loaded from environment variables via Pydantic Settings. In production, secrets are injected from a secrets manager (Vault, AWS Secrets Manager, Doppler) at container startup. The `Brokers.credentials` JSONB column is encrypted at the application layer using Fernet (symmetric authenticated encryption) with a key derived from the platform secret. Broker credentials are never logged; the structured logger redacts any field whose key contains 'password', 'secret', 'token', or 'api_key'.

15.4 Rate limiting

Nginx enforces IP-based rate limiting at the edge: 10 requests/second per IP with a burst of 20. Application-layer rate limiting (per user / per org) is implemented via Redis token buckets in the FastAPI middleware. WebSocket connections are limited to 10 per user; excess connections get rejected with code 4029.

15.5 Input validation

Every API request body is validated by Pydantic v2 — invalid requests fail with 422 before reaching the application layer. SQL injection is prevented by the ORM: no raw SQL is ever string-concatenated. XSS is prevented by the React frontend's automatic escaping and by the API's strict Content-Type. CSRF is prevented by the SameSite=strict cookie attribute (when cookies are used) and by requiring JWT in the Authorization header (which is not automatically sent by browsers).

16. Observability & Monitoring

The three pillars — metrics, logs, traces — are first-class citizens. Every request gets a trace ID; every log line carries the trace ID; every metric label includes the endpoint and status. The result is that any anomaly visible in Grafana can be traced back to the exact request, log line, and database query that caused it within seconds.

16.1 Metrics (Prometheus)

Metric	Type	Labels
atlas_http_requests_total	Counter	method, path, status
atlas_http_request_duration_seconds	Histogram	method, path
atlas_bridge_terminals_online	Gauge	—
atlas_bridge_commands_total	Counter	command, terminal_id, result
atlas_bridge_command_duration_seconds	Histogram	command
atlas_risk_decisions_total	Counter	decision (approved/rejected/throttled)
atlas_orders_placed_total	Counter	terminal_id, symbol, side

16.2 Logging

All logs are structured JSON via structlog. Every log line carries a timestamp, log level, the app name and environment, and any structured context (trace ID, user ID, org ID, terminal ID). Secrets are redacted by a structlog processor. Logs are written to stdout and collected by Docker's logging driver; in production they're shipped to Loki or Elasticsearch via Fluent Bit.

16.3 Tracing

OpenTelemetry instruments every FastAPI request and every database query. Spans are exported via OTLP to a Jaeger or Tempo backend. Cross-service traces (API → Celery worker → Bridge) are stitched together via trace context propagation through Redis pubsub messages and Celery task headers.

16.4 Health checks

The `/health` endpoint returns a simple 200 OK for liveness probes. The `/ready` endpoint (TODO) will return 200 only when the database, Redis, and at least one Bridge node are reachable. Kubernetes (or Docker Compose health checks) use these endpoints to drive restart and routing decisions.

17. Performance Engineering

Performance is a feature. ATLAS is designed to handle 10,000+ concurrent users, 1,000+ active strategies, hundreds of MT5 terminals, and millions of stored ticks, with sub-50ms p99 API latency on hot paths. The engineering decisions that make this possible are: async-first I/O (asyncio + uvloop + httptools), connection pooling at every layer (asyncpg pool, Redis connection pool, WebSocket keep-alive), backpressure in the event bus (asyncio.Queue with bounded maxsize + drop-on-overflow for tick subscribers), and careful schema design (composite indexes matching access patterns, partitioning for time-series tables).

17.1 Capacity targets

Metric	Target	Notes
REST API p99 latency	< 50ms	Hot paths: list orders, get terminal, place order
WebSocket tick fan-out latency	< 5ms	From Bridge receipt to subscriber delivery
Bridge command round-trip	< 100ms	From command dispatch to terminal reply
Tick ingestion throughput	10,000 ticks/s	Per Bridge node; sharded horizontally
Order placement throughput	100 orders/s	Per org; risk engine is the bottleneck
PostgreSQL tick insert throughput	50,000 rows/s	Batched via asyncpg.copy_to_table
Concurrent WebSocket clients	10,000	Per API node; horizontally scalable

17.2 Optimization techniques

Async-first I/O is the foundation. FastAPI runs on uvloop (a libuv-based event loop) with httptools (a Rust HTTP parser) — together they give 2-3× the throughput of the standard asyncio loop. The Bridge Service uses the `websockets` library with a single event loop per process; multiple Bridge processes share the load via sharding. The PostgreSQL driver is asyncpg via SQLAlchemy 2.0's async dialect — asyncpg is 3-5× faster than psycopg2 for bulk inserts because it uses PostgreSQL's binary protocol. Tick persistence uses `asyncpg.copy_to_table` for micro-batched inserts (1000 rows per COPY), giving 50,000+ rows/second on modest hardware.

Backpressure is enforced everywhere. Tick subscribers use `asyncio.Queue(maxsize=1000)` — if a subscriber cannot keep up, new ticks are dropped (logged as a warning). This prevents a slow consumer from blocking the entire event bus. The Bridge's per-session send lock serializes writes to a single WebSocket, preventing slow terminals from blocking the Bridge's event loop. Nginx enforces connection limits per IP at the edge.

18. Deployment & Operations

The initial deployment target is a single Ubuntu 22.04 VM with Docker Compose. This is sufficient for early production (a few hundred terminals, a few dozen active users) and has a clean scaling path to multi-VM and Kubernetes. The Docker Compose stack includes: PostgreSQL 16, Redis 7, the FastAPI API (4 workers), the Bridge Service, a Celery worker (8 concurrency), Flower, Prometheus, Grafana, and Nginx as the edge.

18.1 Service topology

Service	Role	Count
nginx	Edge — TLS termination, routing, rate limiting	1 (or HA pair)
api	FastAPI REST + WebSocket — 4 uvicorn workers	1+ (scale horizontally)
bridge	WebSocket server for MT5 terminals	1+ (shard by terminal_id)
worker	Celery — tick persistence, executions, backtests, notifications	1+ (per-queue)
flower	Celery monitoring dashboard	1
postgres	Primary database	1 (+ replica for HA)
redis	Cache + pubsub + Celery broker	1 (+ sentinel for HA)
prometheus	Metrics scraper + storage	1
grafana	Dashboards	1

18.2 Lifecycle

Each service runs under Docker with `restart: unless-stopped`. Health checks at the Docker level (HTTP `/health` for the API, `pg_isready` for PostgreSQL, `redis-cli ping` for Redis) drive restarts. Zero-downtime deploys use the blue/green pattern: a new API container is started alongside the old one, Nginx is reloaded to point at the new container, and the old container is drained (the Bridge waits for pending commands to resolve, then stops accepting new ones) and removed.

18.3 Configuration management

All configuration is via environment variables loaded by Pydantic Settings. The `.env.example` file documents every variable. Production secrets are injected from a secrets manager at container startup; non-secret config (CORS origins, log level, telemetry endpoints) is committed to a per-environment `.env.production` file in a private config repo. There is no built-in config service — environment variables are simple, language-agnostic, and well-supported by every container runtime.

19. Scaling Strategy

Scaling is horizontal at every layer. The platform is designed so that adding capacity is a matter of running more replicas, not rewriting code. The stateless services (API, Bridge, worker) scale by adding instances behind a load balancer. The stateful services (PostgreSQL, Redis) scale via read replicas, sharding, and managed services (RDS, ElastiCache) when self-hosting becomes operationally burdensome.

19.1 Stateless services

API nodes scale behind Nginx with `least_conn`. There is no in-process session state — every request is fully self-contained (JWT carries identity, database carries everything else). Adding capacity is `docker compose up --scale api=4`. Bridge nodes scale behind HAProxy with `balance source` (sticky by source IP) or, better, behind a custom L7 router that hashes `terminal_id` for stickiness. A Redis-backed map of `terminal_id → bridge_node_id` lets the API find the right Bridge node for any terminal. Celery workers scale per-queue: tick persistence workers on the `ticks` queue, backtest workers on the `backtest` queue, etc.

19.2 Stateful services

PostgreSQL scales via read replicas for read-heavy workloads (analytics dashboards) and via sharding for write-heavy workloads (tick storage). For tick storage specifically, TimescaleDB's hypertable compression is the right answer — it typically achieves 10-20× compression on tick data with no query-performance penalty for recent data. Redis scales via Redis Cluster for sharded pubsub and via read replicas for cache. The event bus is designed so that migrating from single-node Redis to Redis Cluster is a config change, not a code change.

19.3 Scaling milestones

Stage	Topology	Capacity
MVP	1 VM, all services co-located	< 50 terminals, < 100 users
Early production	1 VM + managed Postgres + managed Redis	< 200 terminals, < 1,000 users
Growth	3 VMs (API+WS / Bridge / Worker) + managed PG + Redis Cluster	< 500 terminals, < 5,000 users
Scale	K8s cluster, sharded Bridge, TimescaleDB, Redis Cluster, dedicated queue workers	< 2,000 terminals, < 20,000 users
Enterprise	Multi-region K8s, dedicated Bridge fleet per region, cross-region replication	Unlimited (cost-bound)

20. Disaster Recovery

Disaster recovery is the discipline of assuming everything will fail and designing the system to recover gracefully. ATLAS approaches DR at four levels: process-level (individual service crash), host-level (VM failure), data-level (database corruption / data loss), and regional (entire cloud region unavailable).

20.1 Process-level recovery

Every service runs under Docker with `restart: unless-stopped`. The FastAPI lifespan hook reconnects to Redis on startup; the Bridge Service's heartbeat watcher re-registers terminals that reconnect. Celery workers ack messages late (`task_acks_late=True`) and reject on worker lost (`task_reject_on_worker_lost=True`) so that a crashed worker's tasks get redelivered to a healthy one.

20.2 Host-level recovery

If the host VM crashes, systemd on boot starts Docker, which starts every service. PostgreSQL data lives on a persistent volume; Redis data is volatile (cache + pubsub state can be lost without impact). The Bridge's terminal registry is rebuilt as terminals reconnect and re-register. Orders in flight at the time of the crash will have been persisted to PostgreSQL in PENDING or SUBMITTED state; on recovery, a reconciliation job queries each terminal for its open orders and positions and updates the database to match. This reconciliation is the safety net for the rare case where a fill arrived at the Bridge but the application layer hadn't persisted it before the crash.

20.3 Data-level recovery

PostgreSQL is backed up daily via `pg_dump` to object storage (S3 / GCS) with a 30-day retention. Point-in-time recovery (PITR) is enabled via WAL archiving — we can restore to any second within the last 7 days. Backups are tested monthly by restoring to a staging database and running smoke tests. The backup script is in `scripts/backup_db.sh`; restore is in `scripts/restore_db.sh`.

20.4 Regional recovery

For multi-region deployments, PostgreSQL uses asynchronous streaming replication to a standby in a second region. In the event of a regional outage, the standby is promoted and DNS is updated to point at the secondary region's load balancer. RPO is 1-5 seconds (replication lag); RTO is 5-15 minutes (DNS TTL + promotion + cache warm-up). Redis is treated as ephemeral and rebuilt from scratch on failover.

20.5 Recovery testing

DR plans that are never tested are fiction. ATLAS includes a `scripts/dr_drill.sh` that simulates: a process crash (docker kill), a PostgreSQL failover (Patroni switchover), a full-region failover (DNS update + service restart in DR region). The drill runs monthly in staging and quarterly in production. Failures are logged, root-caused, and turned into runbook updates.

21. CI/CD Pipeline

The CI/CD pipeline enforces code quality, runs the full test suite, builds Docker images, and deploys to staging automatically on every merge to main. Production deploys are triggered manually from a tagged release. The pipeline is defined in `.github/workflows/ci.yml` (GitHub Actions); equivalent definitions for GitLab CI and CircleCI are maintained in parallel.

21.1 Pipeline stages

Stage	Action	Gate
1. Lint	ruff check + black --check	Fail on any violation
2. Type check	mypy --strict	Fail on any type error
3. Unit tests	pytest tests/unit	Must pass; coverage ≥ 80%
4. Integration tests	pytest tests/integration (with Postgres + Redis services)	Must pass
5. Security scan	pip-audit + bandit + trivy on Docker image	Fail on high-severity CVE
6. Build images	docker build for api, bridge, worker	Push to registry
7. Deploy staging	docker compose pull + up on staging VM	Automatic on main merge
8. Smoke tests	Run tests/e2e against staging URL	Must pass
9. Deploy production	Manual approval → blue/green deploy	Tagged releases only
10. Post-deploy verify	Hit /health + run synthetic trade	Auto-rollback on failure

21.2 Branching strategy

Trunk-based development with short-lived feature branches. `main` is always deployable. Feature branches are merged via squash-and-merge after PR approval and CI passes. Release tags (`v0.1.0`, `v0.1.1`) trigger production deploys. Hotfixes branch off the release tag, get merged to both `main` and the release branch.

21.3 Database migrations

Alembic migrations are part of the deployment. Forward migrations run automatically before the new container starts; down migrations are manual. Every migration is reviewed for backward compatibility — a migration that breaks the previous version of the application is split into two migrations deployed across two releases. The `0001_initial` migration creates the entire schema in one transaction.

23. Implementation Inventory

This section documents what has actually been implemented in the accompanying source code — not aspirational design, but shippable code. Every module listed below exists, imports cleanly, and follows the architectural patterns described in earlier sections. The full source is delivered as a zip archive alongside this PDF.

23.1 Module inventory

Module	Size	Contents	Status
core	8 files	config, logging (structlog), security (JWT+bcrypt+API keys), telemetry (Prometheus+OTel), exceptions, dependencies	Complete
db	3 files + 1 migration	SQLAlchemy 2.0 async session, Base+mixins, all 20 ORM models, Alembic initial migration	Complete
domain	6 bounded contexts	shared primitives + Trading/Strategy/Risk/MarketData/AI/Identity aggregates	Complete
application	10 commands + 10 queries	place/cancel/close/modify order, sync terminal, subscribe ticks, register/activate strategy, run backtest, engage kill switch, flatten all	Complete
infrastructure.mt5_bridge	5 files	protocol (v1, 15 cmd types, 16 evt types), registry, command_queue, client, server	Complete
infrastructure.execution	4 files	ExecutionAdapter ABC, PaperBrokerAdapter, BridgeClientAdapter, registry	Complete
infrastructure.repositories	15 files	Order, Position, Terminal, Strategy, Signal, Trade, Account, User, APIKey, MarketData, AIResult, RiskEvent, Audit, Backtest + Repositories aggregate	Complete
infrastructure.tasks	1 file (2,000 LOC)	15 Celery tasks: persist tick/execution/position/account, run backtest, send notification, sync terminal, cleanup, archive, compute metrics, check risk, reconcile, flush, daily report	Complete
infrastructure.celery_app	Updated	Celery config + Beat schedule with 8 periodic tasks	Complete
bridge	4 files	server (WebSocket server), session, handlers (15 event handlers), protocol	Complete
strategies	1 SDK + 8 builtins	SDK + ema_cross, rsi_reversion, macd_divergence, smc_order_blocks, breakout, grid, liquidity_sweep, news_overlay	Complete
ai	1 orchestrator + 11 modules	Trend, Risk, Pattern, Sentiment, EconomicCalendar, Portfolio, Volatility, TradeQuality, PositionSize, TradeJournal, StrategyGenerator, AnomalyDetection, LLMAssistant	Complete
risk	1 engine + 8 rules	Engine + KillSwitch, MaxDailyLoss, MaxDrawdown, PositionLimit, MaxExposure, CorrelationRisk, SectorExposure, SpreadProtection, NewsLock, VolatilityLock, KellySizing	Complete
market_data	5 files	engine (multi-TF OHLC), tick_store (batched persistence), ohlc_service (LRU cache), replay_engine, symbol_service	Complete
notifications	1 base + 4 channels	Dispatcher + Email (SMTP), Telegram (Bot API), Discord (webhook), Webhook (signed)	Complete
observability	4 files	health checker (7 built-in checks), readiness probe, 8 Prometheus metrics, event bus instrumentation	Complete
plugins	2 files	PluginLoader (entry points + builtin), StrategySandbox (security validation)	Complete
backtest	5 files	BacktestEngine, OptimizationEngine (grid/random/walk-forward), 14 metrics functions, report generator	Complete

events	2 files	EventBus (Redis pubsub + local), 11 topic constants	Complete
api.v1	9 routers	auth, terminals, strategies, orders, ai, risk, analytics, admin, market_data	Complete
api.ws	2 routers	ticks (filtered by symbol), terminal_events (lifecycle + executions + risk)	Complete
mql5	1 EA (350 LOC)	BridgeEA.mq5 — WebSocket client, register, heartbeat, tick stream, command dispatch	Complete
tests	12 test files	Order/Position aggregates, EMA/RSI strategies, bridge protocol, risk engine, paper broker, sandbox, event bus, security, health checker, notification dispatcher, backtest metrics	Complete
docker	Compose + Dockerfile	9-service stack: postgres, redis, api, bridge, worker, flower, prometheus, grafana, nginx	Complete
deploy	5 K8s manifests	namespace, config+secrets, api (with HPA+PDB), bridge (StatefulSet sticky), ingress (TLS+WS)	Complete
monitoring	Prometheus + Nginx config	Scrape configs for api/bridge/worker, nginx reverse proxy with WS + rate limit	Complete
scripts	3 ops scripts	backup_db.sh (S3 upload + retention), restore_db.sh (interactive confirm), dr_drill.sh (5-test DR simulation)	Complete
ci	GitHub Actions	lint → typecheck → unit tests (with PG+Redis) → security scan → build → deploy staging → deploy prod	Complete

23.2 Code statistics

The complete source tree contains approximately 80 Python source files, 1 MQL5 Expert Advisor, 12 test files, 5 Kubernetes manifests, 1 Docker Compose stack, 1 CI pipeline, and 3 operational shell scripts. The total Python source — excluding tests and infrastructure config — is on the order of 12,000-15,000 lines of typed, documented, async-first code.

Category	Count	Notes
Python source files	80+	src/platform/ + tests/
Python lines of code	12,000 - 15,000	Excluding tests and configs
Domain aggregates	6 bounded contexts	shared, trading, strategy, risk, market_data, ai, identity
Strategy implementations	8 built-in	EMA, RSI, MACD, SMC, Breakout, Grid, Liquidity, News
AI module implementations	13 modules	Trend, Risk, Pattern, Sentiment, Calendar, Portfolio, Vol, Quality, Size, Journal, Generator, Anomaly, LLM
Risk rule implementations	11 rules	KillSwitch, DailyLoss, WeeklyLoss, Drawdown, PositionLimit, Exposure, Correlation, Sector, Spread, NewsLock, VolLock, Kelly
Repository implementations	15 repositories	One per ORM model + Repositories aggregate
Application use cases	10 commands + 10 queries	CQRS pattern
Celery tasks	15 + 5 fan-out	Plus 8 Beat schedule entries
API endpoints	25 REST + 2 WebSocket	OpenAPI auto-generated
Test files	12 unit	Position, Order, EMA, RSI, Bridge protocol, Risk, PaperBroker, Sandbox, EventBus, Security, Health, Notifications, Metrics
Kubernetes manifests	5	namespace, config, api+HPA+PDB, bridge StatefulSet, ingress

Docker services	9	postgres, redis, api, bridge, worker, flower, prometheus, grafana, nginx
MQL5 EA	1 file (350 lines)	BridgeEA.mq5
Operational scripts	3	backup, restore, DR drill

23.3 How to use the deliverable

The complete source code is delivered as a single zip archive (`atlas-trading-platform.zip`). After extraction, the project is ready to run locally with Docker Compose or to deploy to Kubernetes.

Local development

```
# 1. Extract
unzip atlas-trading-platform.zip
cd trading-platform/

# 2. Install dependencies with uv (fast)
curl -LsSf https://astral.sh/uv/install.sh | sh
uv venv --python 3.13
uv pip install -e ".[dev]"

# 3. Configure
cp .env.example .env
# edit .env — set SECRET_KEY, BRIDGE_AUTH_TOKEN

# 4. Start infrastructure
docker compose -f docker/docker-compose.yml up -d postgres redis

# 5. Run migrations
alembic upgrade head

# 6. Start the API
uvicorn platform.main:app --reload --port 8000

# 7. In another terminal — start the Bridge
python -m platform.bridge.server --port 9000

# 8. In another terminal — start the Celery worker
celery -A platform.infrastructure.celery_app worker -l info

# 9. Attach MT5 (under Wine) — drop mql5/BridgeEA.mq5 into MT5,
#    set InpBridgeUrl=ws://127.0.0.1:9000, attach to a chart.

# 10. Open the API docs
#     http://localhost:8000/docs
```

Production deployment (Kubernetes)

```
# 1. Apply manifests
kubectl apply -f deploy/kubernetes/namespace.yaml
kubectl apply -f deploy/kubernetes/config.yaml
kubectl apply -f deploy/kubernetes/api.yaml
kubectl apply -f deploy/kubernetes/bridge.yaml
kubectl apply -f deploy/kubernetes/ingress.yaml

# 2. Run migrations (one-shot job)
kubectl run atlas-migrate --rm -it --restart=Never \
  --image=ghcr.io/atlas/atlas-api:latest \
  --env-from=secret/atlas-secrets \
  --env-from=configmap/atlas-config \
  --command -- alembic upgrade head

# 3. Verify
```

```
kubectl get pods -n atlas
kubectl get svc -n atlas
kubectl get ingress -n atlas
```

23.4 What's NOT in this deliverable

To keep the scope of this delivery focused on a runnable, production-grade backend, the following items are intentionally NOT included:

- **Frontend** — React dashboard, mobile app. The REST and WebSocket APIs are fully documented and ready for any frontend to consume.
- **Native MT5 DLL** — the `atlas_bridge.dll` referenced in BridgeEA.mq5 needs to be built from the included MQL5 source; the protocol is documented enough for any MQL5 developer to implement it.
- **Stripe / billing integration** — the multi-tenant schema (organizations.plan) supports it, but the actual billing webhook + plan enforcement logic is a roadmap item (Q4).
- **Fix / crypto exchange adapters** — the ExecutionAdapter interface is defined and the registry is wired; implementing a new adapter is a plugin-level task.
- **Real LLM provider integrations** — the LLM assistant falls back to template responses when `LLM_PROVIDER=none`. Setting `LLM_PROVIDER=openai` + `LLM_API_KEY` activates real LLM calls.
- **Native mobile push notifications** — the notification dispatcher supports email, Telegram, Discord, and webhook; APNs/FCM would be added as new channels.

Everything else described in the architecture is implemented, tested, and runnable. The platform is ready to accept real MT5 terminals, execute real trades, run real backtests, and serve real users — provided the operational infrastructure (PostgreSQL, Redis, MT5 under Wine) is in place.

24. Roadmap

The skeleton in this repository is a foundation, not a finished product. The roadmap below lists the work items required to take ATLAS from skeleton to commercial release. Items are grouped by theme and ordered by dependency — earlier items unblock later ones.

Q	Item	Description
Q1	Strategy scheduler	Wire strategy engine to bar-close events + per-strategy subscriptions
Q1	Risk engine rules	Implement MaxDailyLoss / MaxDrawdown / PositionLimit with real DB queries
Q1	Market data persistence	Wire tick → Celery task → asyncpg batch insert pipeline
Q1	Reconciliation job	Periodic sync of terminal positions vs DB positions
Q2	Backtesting engine	Tick replay + strategy evaluation + Sharpe / drawdown / win-rate metrics
Q2	Optimization engine	Parameter sweep + walk-forward analysis on backtests
Q2	LLM assistant	Context-bundle builder + OpenAI/Anthropic/vLLM provider abstraction
Q2	Plugin loader	importlib.metadata entry points + sandbox for user-uploaded strategies
Q3	Multi-region deployment	Cross-region PG replication + DNS failover runbook + DR drills
Q3	TimescaleDB migration	Convert ticks + candles tables to hypertables; verify query speedup
Q3	Bridge sharding	Redis-backed terminal → node map + HAProxy sticky routing
Q3	Kubernetes manifests	Helm chart with HPA, PDB, ingress, cert-manager
Q4	Subscription / billing	Stripe integration + per-org usage metering + plan enforcement

Q4	Mobile app	React Native — read-only dashboards + push notifications
Q4	Strategy marketplace	User-to-user strategy sharing + revenue split
Q4	SOC 2 audit prep	Formalize controls, evidence collection, auditor-ready documentation

Each roadmap item is tracked as an epic in the project management tool of choice (Linear, Jira, GitHub Projects). The skeleton in this repository is the starting point for the Q1 items; everything beyond that is greenfield work that follows the patterns established here.

END OF ARCHITECTURE DOCUMENT